

Parcel Risk Report — Technical Design Document

Sep 26, 2026 · @Fa

Overview

Parcel Risk Report turns an address into a cited, satellite-backed climate-risk report in under 30 seconds. It covers five hazards: flooding, ground subsidence, wildfire, extreme heat, and nearby land-use change.

Problem. Buyers, lenders, and insurers price climate risk from flood zone maps and generic scores. Those maps are often outdated, and the scores rarely show their evidence. Free satellite archives already record where it flooded, where the ground is sinking, and where it burned, but turning that into a per-property answer takes remote sensing expertise most buyers lack.

Goals

- Return a report for any address in the pilot region in under 30 seconds (p95).
- Score each hazard with a calibrated confidence, not just a label.
- Cite the specific observation behind every finding (sensor, date, value).
- Keep marginal cost per report low enough for a consumer price point.

Non-goals (v1)

- Forward-looking climate projections (2050 scenarios). v1 reports observed history only.
- Insurance pricing or a formal appraisal. The report informs decisions; it does not replace a licensed assessment.
- Global coverage. v1 targets one US metro, where subsidence products and parcel data exist.
- Running interferometry or training new flood models in-house.

Target users

User	Question they ask	What they need from the report
Home buyer	Is this house safe to buy?	Plain-language summary, clear red flags

User	Question they ask	What they need from the report
Lender or underwriter	Does this collateral carry hidden risk?	Per-hazard scores, confidence, audit trail
Insurer	How should this property be priced?	Observation history, comparison to area baseline

Requirements

The 30-second budget drives the whole design: no satellite imagery is processed at request time.

Functional

1. Accept a street address, geocode it, and resolve it to a parcel polygon (fall back to a buffer around the point when no parcel exists).
2. Compute per-hazard features for the parcel and a surrounding buffer (default 500 m).
3. Score each hazard: risk level, 0–100 score, and a confidence value.
4. Generate a readable report where every claim links to its evidence (source, date range, measured value).
5. Show a map with each hazard layer toggleable over the parcel.
6. Export the report as a PDF and a JSON payload for programmatic users.
7. Flag when data is missing or stale instead of silently scoring it as low risk.

Non-functional

Requirement	Target
End-to-end latency	p95 under 30 s, p50 under 10 s
Cached report latency	Under 1 s
Availability	99.5% for the API (pilot)
Cost per uncached report	Under \$0.05 in compute and model calls
Traceability	100% of findings cite a stored observation
Reproducibility	Same address + same data version = same scores

Constraints

- Solo developer or a small team; favor managed services.

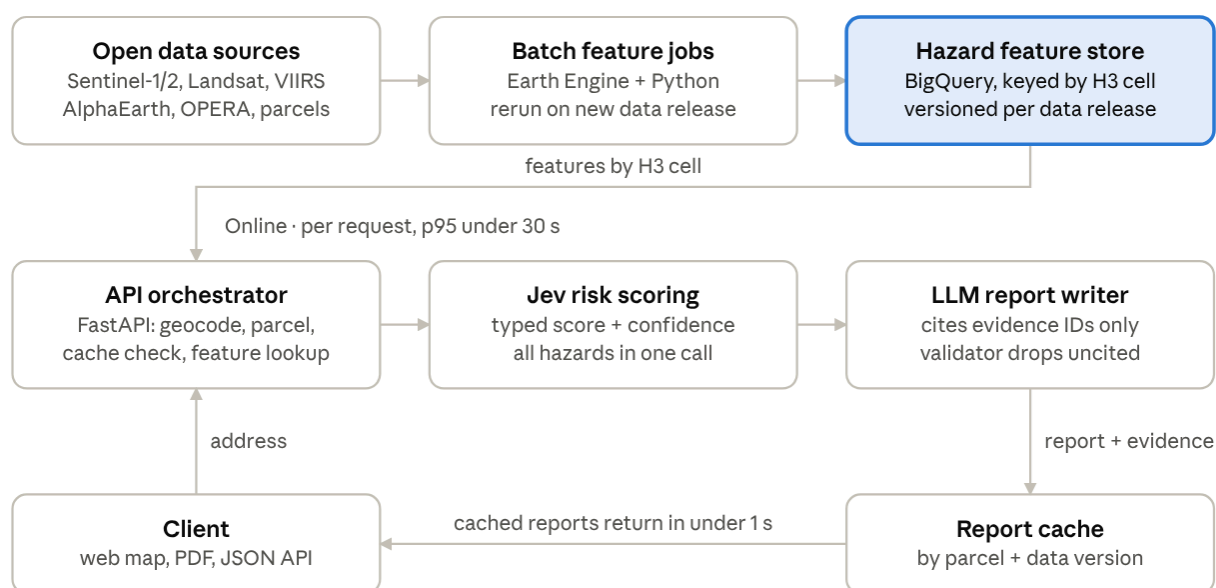
- Free or open data only in v1 (Copernicus, NASA, USGS, Google Earth Engine, public parcel data).
- Stack builds on existing experience: Python, FastAPI, TypeScript, React, BigQuery, Earth Engine.

High-level architecture

All satellite processing happens offline; a request only reads precomputed features, scores them, and writes the report.

Imagery is processed offline; requests only look up features

Offline · batch refresh when new data lands



system architecture · offline batch path and online request path

The offline path refreshes the feature store when a source publishes new data. The online path touches only the store, Jev, and the LLM, which keeps it inside the 30-second budget.

Data sources

Each hazard uses processed, open products where they exist; the only in-house detection is flood mapping, and even that starts from an existing archive.

Hazard	Primary source	Resolution	Update cadence	Feature derived	Main limitation
Flood	Sentinel-1 radar flood extents (Copernicus Global Flood Monitoring archive, or own detection on event dates)	~20 m	6-day revisit	Times flooded since 2015, last flood date, share of parcel flooded	Misses short floods between passes; urban flooding is hard to detect in radar
Subsidence	NASA OPERA surface displacement product (Sentinel-1 interferometry)	~30 m	Per new product release	Vertical velocity (mm/yr) and trend vs. area average	Coverage limited to North America; verify for pilot metro
Wildfire	VIIRS active fire archive (NASA FIRMS) + MTBS burn perimeters	375 m / ~30 m	Daily / annual	Fires within 1, 5, 10 km since 2012; nearest past burn perimeter	Coarse points; small fires missed
Heat	Landsat 8/9 land surface temperature, summer composites	100 m native, delivered at 30 m	8 days combined, cloud-limited	Mean summer surface temp and difference from metro median	Surface temp, not air temp; one overpass time (~10:30 am)
Land-use change	AlphaEarth annual embeddings; Sentinel-2 to confirm	10 m	Annual (2017–2025)	Share of 500 m buffer changed year over year; change type (built, cleared, wetland loss)	Annual only; change type needs a trained classifier
Context	County parcel polygons, Census geocoder, FEMA flood zones	Parcel	Varies by county	Parcel geometry; official flood zone for comparison	Parcel data format differs per county

Figures are approximate and come from earlier research plus general knowledge; confirm each product's coverage and license for the pilot region before building.

Processing pipeline

Every hazard is reduced offline to numbers per H3 cell (resolution 10, about 0.015 km² per cell), each carrying its provenance, so a request never touches raw imagery.

Offline jobs (one per hazard, run when a source publishes new data)

1. **Flood:** pull archived Sentinel-1 flood extents for the pilot area; for each cell, count flooded observations and record the last flood date. Where no archive exists, run a threshold-based water detection on radar scenes from known storm dates.
2. **Subsidence:** sample OPERA displacement rasters at cell centroids; fit a linear velocity per cell and store the fit error alongside it.
3. **Wildfire:** load the FIRMS VIIRS archive and MTBS perimeters; compute fire counts within 1, 5, and 10 km of each cell and distance to the nearest past burn.
4. **Heat:** build a cloud-masked June–August Landsat surface temperature composite per year in Earth Engine; store the cell mean and its difference from the metro median.
5. **Land-use change:** compute cosine similarity between consecutive AlphaEarth years per 10 m pixel; flag pixels below a tuned threshold and classify the change type with a small classifier trained on labeled examples.

Each job writes rows keyed by `(h3_cell, hazard, data_version)` and an evidence record listing the source product, scene or granule IDs, date range, and method version.

Online request path

1. Geocode the address (Census geocoder) and fetch the parcel polygon; fall back to a 30 m point buffer.
2. Check the report cache by `(parcel_id, data_version)`; return on hit.
3. Cover the parcel and a 500 m buffer with H3 cells; aggregate each hazard's features by area weight.
4. Build one JSON state per request: features, area baselines, data gaps, and evidence IDs.
5. Call Jev once with all hazard questions; call the LLM to write the report from the scores and evidence.
6. Validate citations, render the PDF, write the cache, and return.

Missing data is explicit. A cell with no usable observations gets a `no_data` flag, never a zero, and the report says so.

AI layer

Code computes every number, Jev makes the judgment calls on those numbers, and the LLM only writes prose around facts it is given.

Jev risk scoring

Jev receives the request's JSON state and a fixed set of typed questions, all answered in one parallel call. Each question asks one narrow thing, following TypeSafe's guidance to decompose multi-factor judgments.

Question	Type	Answers
flood_level , subsidence_level , wildfire_level , heat_level , land_change_level	Choice	low · moderate · high · severe, each with written criteria
<hazard>_material_to_buyer	Noul (yes/no probability)	Would a typical buyer want to know this before purchase?
land_change_type	Choice	new development · clearing · wetland loss · none
needs_expert_review	Noul	Do the signals conflict or fall outside normal ranges?

The 0–100 hazard score is computed in code from the level probabilities (probability-weighted midpoint), so it stays reproducible and Jev never does arithmetic.

Confidence policy

- Confidence at or above a tuned threshold: the level is shown as the finding.
- Below threshold: the report shows the level as "uncertain," lists the conflicting evidence, and queues the report for review.
- A deterministic rule baseline scores every hazard too. It is the fallback if Jev is unavailable, and the comparison point in evaluation.

Pin `jev-1.13.0`, not `jev-latest`. A model update can shift answers, so thresholds are re-tuned before any version bump.

LLM report writer

The LLM receives the Jev outputs, the features, and an evidence list with IDs (`E1` , `E2` , ...). Instructions require every factual sentence to end with an evidence ID and forbid numbers not present in the evidence.

A validator checks the draft before it ships:

1. Every sentence with a number or hazard claim cites at least one evidence ID.
2. Every cited ID exists, and every number in the sentence matches a value in that evidence record.
3. The stated risk levels match Jev's output.

On failure, regenerate once with the errors attached; on a second failure, fall back to a templated report built directly from the scores. Every report ends with a fixed disclaimer that it reflects observed satellite history, not a formal assessment.

Data model and API

Four tables and three endpoints cover v1; every score traces back through `evidence_id` to a source granule.

Tables (BigQuery)

Table	Key	Main columns
<code>hazard_features</code>	<code>h3_cell</code> , <code>hazard</code> , <code>data_version</code>	<code>metrics</code> (JSON), <code>no_data</code> (bool), <code>evidence_ids</code>
<code>evidence</code>	<code>evidence_id</code>	<code>source_product</code> , <code>granule_ids</code> , <code>date_start</code> , <code>date_end</code> , <code>method_version</code> , <code>values</code> (JSON)
<code>parcels</code>	<code>parcel_id</code>	<code>county_fips</code> , <code>geometry</code> (GEOGRAPHY), <code>h3_cells</code> , <code>buffer_cells</code>
<code>reports</code>	<code>report_id</code>	<code>parcel_id</code> , <code>data_version</code> , <code>jev_version</code> , <code>llm_model</code> , <code>scores</code> (JSON), <code>report_md</code> , <code>created_at</code>

The report cache is a Redis key `report:{parcel_id}:{data_version}` pointing at `report_id`; `reports` is the durable copy and the audit log.

Endpoints (FastAPI)

Method and path	Purpose
<code>POST /v1/reports</code>	Body <code>{ "address": "..."} </code> . Returns the report JSON; streams progress events when requested.
<code>GET /v1/reports/{report_id}</code>	Fetch a stored report as JSON.
<code>GET /v1/reports/{report_id}/pdf</code>	Download the rendered PDF.

Report response (abridged)

```
{
  "report_id": "rpt_01J...",
  "parcel_id": "25013-...",
  "data_version": "2026-09",
  "hazards": {
    "flood": {
      "level": "high",
```

```
    "score": 72,
    "confidence": 0.84,
    "material_to_buyer": 0.91,
    "evidence": ["E1", "E2"]
  },
  "needs_expert_review": false,
  "data_gaps": ["subsidence: outside product coverage"],
  "evidence": {
    "E1": {"source": "Sentinel-1 flood extent", "dates": "2015-2026",
"value": "flooded in 3 of 11 events"}
  },
  "report_md": "...
}
```

Performance, caching, and cost

The LLM call dominates both latency and cost; everything else fits in a few seconds and fractions of a cent.

Latency budget (uncached request, approximate)

Step	Target
Geocode + parcel lookup	0.5–1 s
Feature aggregation from the store	1–3 s
Jev scoring (one call, all questions)	under 0.5 s
LLM report generation	5–15 s
Validation, PDF render, cache write	1–2 s
Total	~8–22 s, inside the 30 s p95

Stream progress events to the client ("Checking flood history...") so the wait feels shorter, the same pattern as Biz Mapper's Thinking Log.

Caching

- Reports cache by `(parcel_id, data_version)`; a new data release invalidates them naturally.
- Pre-warm the cache for parcels with active listings in the pilot metro.

- If BigQuery's per-query overhead eats too much of the budget, mirror the pilot's feature rows into Postgres/PostGIS or Redis; at pilot scale that is only a few hundred thousand rows (about 66,000 cells per 1,000 km² × 5 hazards).

Cost per uncached report (approximate)

- Jev: a few thousand input tokens at TypeSafe's listed \$42 per billion → well under \$0.001.
- LLM: roughly 6,000 input and 1,500 output tokens → about \$0.01–0.03, depending on model.
- BigQuery reads on a clustered table and geocoding → near zero at pilot volume.

Offline jobs run in Earth Engine and batch Python; for one metro they should fit comfortably within free or academic quotas, to be confirmed in milestone 1.

Validation and evaluation

Each hazard is checked against an independent ground-truth source, and the scoring layer must beat the rule baseline before Jev ships as the default.

Hazard features vs. independent data

Hazard	Ground truth	Check
Flood	OpenFEMA flood insurance claims (area-level), FEMA flood zones, news-documented flood events	Do high-flood cells cluster where claims and documented events are? Where do satellite and FEMA maps disagree?
Subsidence	GNSS station velocities in the pilot area	Velocity error at station locations
Wildfire	State fire perimeters not used as inputs	Share of known burns within the scored high-risk area
Heat	NOAA community heat-mapping campaign data or local weather stations, where available	Rank correlation between surface temp and measured air temp
Land-use change	Hand-labeled sample of ~300 locations checked in NAIP aerial imagery	Precision and recall of changed pixels and change type

Scoring layer

- Build a labeled set of about 300 parcels, with levels assigned by hand from the evidence (ideally by someone with planning or insurance experience).
- Compare Jev and the rule baseline on accuracy and calibration: when Jev says 80% confident, it should be right about 80% of the time.
- Tune the confidence threshold on a held-out split; report how many reports it routes to review.

Reports

- The citation validator must pass on 100% of shipped reports.
- Hand-review 50 reports for faithfulness: no claim beyond the evidence, no missing red flag.
- Load-test the pilot at expected peak traffic against the 30 s p95 target.

Risks and trade-offs

The biggest risk is false reassurance: a clean-looking report for a property the satellites simply could not see well.

Risks

Risk	Impact	Mitigation
Missing data read as low risk	Buyer skips a real hazard	<code>no_data</code> flags, a data-gaps section in every report, never score gaps as zero
Satellites miss short or urban floods	Flood risk understated	State the limit in the report; show FEMA zone beside satellite history
LLM invents or distorts a claim	Wrong or misleading report	Citation and number validator; templated fallback
Jev is in early access; answers shift between versions	Scores drift or the API changes	Pin the version; rule baseline as fallback; re-tune before upgrades
Report affects property value or lending decisions	Legal and fair-housing exposure	Hazard data only, no demographic inputs in scoring; fixed disclaimer; legal review before any paid launch

Risk	Impact	Mitigation
Data licenses	Redistribution limits	Track license per source; include required attribution (AlphaEarth is CC-BY 4.0); check county parcel terms

Key design decisions

Decision	Chosen	Alternative	Why
When to process imagery	Offline precompute	On demand per request	Only way to meet 30 s; results are reproducible per data version
Spatial unit	H3 cells, aggregated to parcel	Raw parcel polygons	One index for all hazards; fast joins; buffers are simple rings
Risk judgment	Jev + rule baseline	LLM-only or rules-only	Calibrated confidence per hazard; rules keep a fallback and a benchmark
Feature store	BigQuery	Postgres/PostGIS	Familiar and scales; revisit if query overhead hurts latency
Subsidence	Use OPERA products	Own interferometry	Interferometry is a research project on its own

Milestones and open questions

The MVP is one US metro, five hazards, and a web report; each phase must pass its gate (diamond) before the next begins.

Four phases, each gated on validation before the next starts



MVP roadmap · 4 phases, each ending in a validation gate

Phase 3 (highlighted) carries the most risk, since it decides whether Jev earns its place over the rule baseline.

Open questions

- ☐ Which pilot metro? It needs OPERA subsidence coverage, open parcel data, and ideally a history of floods or fires to validate against. Austin, TX
- ☐ Use the Copernicus flood archive, or build own Sentinel-1 flood detection? Build own sentinel-1 flood detection
- ☐ Which LLM for the report writer, given the cost and latency targets? gemini 3.6 flash
- ☐ Is Jev early access available for this use, and on what terms? yes
- ☐ Who labels the 300-parcel scoring set, and can someone with insurance or planning experience review it? no, this is just a proof of concept (a prototype)
- ☐ Consumer product first, or an API for lenders and insurers? consumer product first, but also api-ready